

Lab 1 — SNMP and Python

Network Discovery, Device Inspection, and Monitoring

Álvaro Alesanco Iglesias

Table of contents

1	Context	3
2	Learning Objectives	3
3	Lab Environment	4
4	Preparing the Environment	5
4.1	Accessing the Debian Container	5
4.2	Installing SNMP Command-Line Tools	5
4.3	Creating a Python Virtual Environment	6
4.4	Installing PySNMP	7
4.5	Checking Network Connectivity	7
5	First Contact with SNMP	8
5.1	Configuring the MikroTik SNMP Agent	8
5.2	Your First SNMP GET	9
5.3	Exploring the <code>system</code> Group	10
5.4	Looking at the Protocol	11
5.5	Performing the Same GET with Python	12
6	GET Operations and VarBinds	12
6.1	A Minimal PySNMP Example	13
6.2	Retrieving the <code>system</code> Objects	14
6.2.1	Experiment 1 — One GET per object	15
6.2.2	Experiment 2 — Several VarBinds in one GET	15
6.3	Verifying the Difference on the Network	16
6.4	What Happens When an Object Does Not Exist?	17
6.4.1	SNMPv1 vs SNMPv2c	17
6.5	Sequential Requests vs Concurrent Requests	18

7	Navigating the MIB: GETNEXT and WALK	19
7.1	Using GETNEXT	19
7.2	Building a WALK from GETNEXT	20
7.2.1	Experiment — Implement your own walk	21
7.3	Comparing with <code>snmpwalk</code>	22
8	Working with SNMP Tables	22
8.1	The Interface Table	22
8.2	Building an Interface Inventory	24
8.3	From Inventory to Telemetry	24
9	From Counters to Telemetry	25
9.1	From a Counter to a Rate	25
9.2	Measuring the Traffic Rate of an Interface	26
9.3	Sampling Interval	27
9.4	Continuous Monitoring	27
9.5	Counter Size Matters	28
9.6	Using High-Capacity Counters	29
9.7	Interface Utilization	30
10	Monitoring More Than One Interface	30
10.1	Concurrent Polling with <code>asyncio</code>	31
10.2	Final Monitor	32
11	Automatic Device Discovery	33
11.1	Discovering Hosts on a Local Network	33
11.2	ARP Discovery with Scapy	34
11.2.1	Building Packets with Scapy	35
11.2.2	Ethernet and ARP	35
11.2.3	Sending and Receiving Packets	36
11.2.4	Discovering a Complete Subnet	38
11.2.5	Exercise — Build the Host Inventory	39
11.3	ARP Is Not the Only Discovery Mechanism	40
11.4	From a Host to a Managed Device	40
11.5	Identifying SNMP Devices	41
11.6	Enterprise OIDs	42
11.7	Why Sequential Discovery Does Not Scale	43
11.8	Concurrent SNMP Discovery	43
11.9	Building the Device Inventory	44
12	Modifying Managed Objects: SNMP SET	45
12.1	Read and Write Access	46
12.2	Enabling Write Access	46

12.3 Performing a SET with PySNMP	47
12.4 Inspecting an SNMP SET	48
12.5 Monitoring vs Configuration	49
13 Securing SNMP: SNMPv3	49
13.1 The Problem with Community-Based SNMP	50
14 SNMPv3 Security Model	51
14.1 Configuring an SNMPv3 User	52
14.2 Your First SNMPv3 GET	52
14.3 Inspecting SNMPv3 on the Network	53
14.4 SNMPv3 with PySNMP	54
15 Comparing SNMPv2c and SNMPv3	55
16 What Have We Built?	56

1 Context

SNMP (Simple Network Management Protocol) remains one of the most widely used mechanisms for retrieving status and operational information from network devices.

In this lab, we will use a MikroTik router as the managed device and Python to develop simple network management and monitoring tools.

The lab follows a progression similar to that used by real network management platforms:

1. **Discover** which devices are present in the network.
2. **Identify** which devices can be managed using SNMP.
3. **Inspect** the information exposed by those devices.
4. **Explore** their interfaces and other objects available through their MIBs.
5. **Monitor** how selected network parameters evolve over time.

Throughout the lab, you will work directly from a Linux system and use both command-line tools and Python scripts.

2 Learning Objectives

After completing this lab, you should be able to:

- prepare a Python environment on Linux for developing simple network management tools;
- understand the relationship between **OIDs**, **MIBs**, scalar objects, and tables;
- perform SNMP queries using **GET**, **GETNEXT**, and **WALK**;

- interpret SNMP responses and the **VarBind** structures exchanged between manager and agent;
- modify managed objects using **SET** when permitted by the agent;
- automatically discover devices on a network and determine which of them provide SNMP management;
- retrieve and interpret the interface table of a network device;
- use SNMP counters to calculate traffic rates;
- develop a simple interface monitor using Python;
- understand the fundamental differences between SNMPv1, SNMPv2c, and SNMPv3.

3 Lab Environment

Each student has an individual laboratory environment accessible through the CyberRange.

The lab will mainly use two systems:

- a **Debian container**, from which command-line tools and Python scripts will be executed;
- a **MikroTik router**, which will act as the SNMP agent and the network device being monitored.

The basic architecture is:

```

Debian CT

Python / pysnmp
snmpget / snmpwalk
tcpdump

SNMP

MikroTik

SNMP Agent
MIB-II

```

4 Preparing the Environment

In previous labs, the programming environment and the required Python libraries may have been provided preconfigured. In this course, you will prepare your own working environment.

This is a common task when deploying network management and observability tools: before collecting telemetry, you need a system from which to run them.

4.1 Accessing the Debian Container

Connect to your **Debian container (d13-basic-153)** through the CyberRange and open a terminal.

Check the operating system and the installed Python version:

```
cat /etc/os-release  
python3 --version
```

Throughout this lab, the Debian container will act as the **SNMP manager**, while the MikroTik router will act as the **SNMP agent**.

Manager and agent

An **SNMP manager** sends requests to managed devices and processes their responses.
An **SNMP agent** runs on the managed device and provides access to the management information exposed by that device.

4.2 Installing SNMP Command-Line Tools

Before writing Python programs, we will verify SNMP communication using standard Linux command-line tools.

Update the package information and install the SNMP utilities:

```
sudo apt update  
sudo apt install snmp
```

Check that the tools are available:

```
snmpget --version
```

Some of the commands that we will use during this lab are:

Command	Purpose
<code>snmpget</code>	Retrieve one or more specific objects
<code>snmpgetnext</code>	Retrieve the next object in the MIB tree
<code>snmpwalk</code>	Traverse a subtree of the MIB
<code>snmpset</code>	Modify a writable object

Do not worry about their complete syntax yet. We will introduce each command when it is needed.

4.3 Creating a Python Virtual Environment

Python projects commonly use **virtual environments** to keep their dependencies isolated from the operating system and from other projects.

Install the required package if it is not already available:

```
sudo apt install python3-venv
```

Create a directory for the lab:

```
mkdir ~/p1-snmp  
cd ~/p1-snmp
```

Create a virtual environment:

```
python3 -m venv .venv
```

Activate it:

```
source .venv/bin/activate
```

Your shell prompt should now indicate that the virtual environment is active.

Tip

Whenever you start a new terminal and return to this lab, activate the environment again:

```
cd ~/p1-snmp
source .venv/bin/activate
```

4.4 Installing PySNMP

With the virtual environment activated, install PySNMP:

```
pip install pysnmp
```

Verify the installation:

```
python3 -c "import pysnmp; print(pysnmp.__version__)"
```

Create a file named `test.py`:

```
import pysnmp

print("PySNMP version:", pysnmp.__version__)
```

Run it:

```
python3 test.py
```

If the version is displayed without errors, your Python environment is ready.

4.5 Checking Network Connectivity

Before using SNMP, verify that the MikroTik router is reachable from your Debian container.

```
ping -c 3 <MIKROTIK_IP>
```

Replace `<MIKROTIK_IP>` with the IP address assigned to the MikroTik router in your laboratory environment.

! Important

Being able to reach a device using ICMP does **not** imply that the device supports SNMP. Similarly, a device that does not reply to ICMP Echo Requests could still provide other network services.

We will revisit this distinction when implementing automatic device discovery.

At this point, your environment should contain:

Debian CT

Linux SNMP tools

`snmpget`

`snmpgetnext`

`snmpwalk`

`snmpset`

`~/p1-snmp/`

`.venv/`

`test.py`

You are now ready to communicate with the MikroTik SNMP agent.

5 First Contact with SNMP

We now have the tools required to act as an SNMP manager. The next step is to configure the MikroTik router as an SNMP agent and perform our first query.

5.1 Configuring the MikroTik SNMP Agent

Connect to your MikroTik router and verify the SNMP configuration.

Enable the SNMP agent if necessary and configure the following community:

Parameter	Value
SNMP version	SNMPv2c
Community	<code>public</code>
Access	Read-only

Warning

SNMPv1 and SNMPv2c communities are transmitted without encryption and should not be considered secure authentication mechanisms.

We use SNMPv2c here because it allows us to focus first on the SNMP data model and protocol operations. SNMPv3 will be introduced later in the lab.

5.2 Your First SNMP GET

Let's retrieve `sysDescr.0` from the MikroTik.

`sysDescr` belongs to the `system` group of MIB-II and contains a textual description of the managed device.

Its numeric OID is:

```
1.3.6.1.2.1.1.1.0
```

From the Debian container, execute:

```
snmpget -v2c -c public <MIKROTIK_IP> 1.3.6.1.2.1.1.1.0
```

You should obtain a response similar to:

```
iso.3.6.1.2.1.1.1.0 = STRING: ...
```

The exact value depends on your MikroTik router.

Notice that the result contains three different pieces of information:

```
1.3.6.1.2.1.1.1.0 = STRING : value
                        Object value
                        ASN.1 data type
                        Object identifier
```

This (OID, `value`) association is the basic information carried by an SNMP **VarBind**.

i Why does the OID end in .0?

`sysDescr` is a scalar object.

SNMP identifies an **instance** of a scalar object by appending .0 to the object OID. Therefore, a GET request must use the complete instance OID:

```
sysDescr.0
```

Later, when working with tables, the suffix will have a different meaning: it will identify a particular row of the table.

5.3 Exploring the system Group

The `system` group is located below:

```
1.3.6.1.2.1.1
```

Use `snmpwalk` to retrieve the objects available below this OID:

```
snmpwalk -v2c -c public <MIKROTIK_IP> 1.3.6.1.2.1.1
```

Identify at least the following objects in the output:

Object	Information
<code>sysDescr</code>	Device description
<code>sysObjectID</code>	Type of managed device
<code>sysUpTime</code>	Time since the management system was initialized
<code>sysContact</code>	Contact information
<code>sysName</code>	Device name
<code>sysLocation</code>	Physical/location information
<code>sysServices</code>	Services provided by the device

Compare `sysDescr` and `sysObjectID`.

What type of information does each object provide? Why could `sysObjectID` be particularly useful for an automatic network management system?

5.4 Looking at the Protocol

So far, the `snmpget` command has hidden the actual SNMP message exchange.

Capture the traffic generated by a single GET operation using the packet capture mechanism provided in the laboratory environment.

Start the capture and execute again:

```
snmpget -v2c -c public <MIKROTIK_IP> 1.3.6.1.2.1.1.1.0
```

Stop the capture after receiving the response and inspect the SNMP packets.

You should find two messages:

Debian CT (manager)	MikroTik (agent)
----- GET request ----->	
<----- RESPONSE -----	

Inspect the GET request and answer the following questions:

1. Which SNMP version is being used?
2. Can you find the community string in the packet?
3. Which OID appears in the VarBind?
4. What value is included in the VarBind of the GET request?
5. Find the corresponding VarBind in the response. What has changed?

! Important

Do not capture and analyse dozens of packets. The objective is to understand **one request and its corresponding response**.

You should be able to relate what you see in the packet capture to the SNMP manager/agent model studied in class.

5.5 Performing the Same GET with Python

The command-line tools are useful for testing and troubleshooting, but our goal is to build network management software.

Create a file named:

```
get_system.py
```

Using PySNMP, retrieve `sysDescr.0` from your MikroTik router.

Your program should display:

```
OID:    ...
Type:   ...
Value:  ...
```

Tip

At this point, focus on understanding the PySNMP operation rather than building a generic SNMP application.

Identify in your program:

- the SNMP version;
- the community;
- the address of the agent;
- the requested OID;
- the returned VarBind.

Once the program works, compare its result with the one obtained using `snmpget`.

Both tools are acting as SNMP managers and should retrieve the same managed object from the same agent.

6 GET Operations and VarBinds

In the previous section, you retrieved a single managed object using an SNMP GET operation.

An SNMP GET request, however, can contain **more than one VarBind**. This allows a manager to request several objects using a single SNMP message.

In this section, we will compare both approaches and examine how SNMP handles errors.

6.1 A Minimal PySNMP Example

PySNMP provides an asynchronous API based on Python's `asyncio`.

The following example retrieves `sysUpTime.0` from an SNMP agent:

```
import asyncio

from pysnmp.hlapi.v3arch.asyncio import (
    SnmpEngine,
    CommunityData,
    UdpTransportTarget,
    ContextData,
    ObjectType,
    ObjectIdentity,
    get_cmd,
)

async def main():
    target = await UdpTransportTarget.create(
        ("<MIKROTIK_IP>", 161)
    )

    error_indication, error_status, error_index, var_binds = await get_cmd(
        SnmpEngine(),
        CommunityData("public", mpModel=1),
        target,
        ContextData(),
        ObjectType(ObjectIdentity("1.3.6.1.2.1.1.3.0")),
    )

    if error_indication:
        print(error_indication)
    elif error_status:
        print(
            f"{error_status.prettyPrint()} at "
            f"{error_index and var_binds[int(error_index) - 1][0] or '?'}"
        )
    else:
        for oid, value in var_binds:
            print("OID: ", oid.prettyPrint())
            print("Type: ", type(value).__name__)
```

```
print("Value:", value.prettyPrint())

asyncio.run(main())
```

Replace <MIKROTIK_IP> with the address of your MikroTik router and execute the program.

Before continuing, identify in the program:

- where the SNMP agent is specified;
- where the UDP port is specified;
- where the community is specified;
- where SNMPv2c is selected;
- where the requested OID is specified;
- where the program waits for the SNMP response;
- which variable contains the returned VarBinds.

Why is `await` appropriate for an operation such as an SNMP request?

6.2 Retrieving the system Objects

Create a new program named:

```
get_system.py
```

The program will retrieve the following objects:

Object	OID
sysDescr.0	1.3.6.1.2.1.1.1.0
sysObjectID.0	1.3.6.1.2.1.1.2.0
sysUpTime.0	1.3.6.1.2.1.1.3.0
sysContact.0	1.3.6.1.2.1.1.4.0
sysName.0	1.3.6.1.2.1.1.5.0
sysLocation.0	1.3.6.1.2.1.1.6.0
sysServices.0	1.3.6.1.2.1.1.7.0

6.2.1 Experiment 1 — One GET per object

Retrieve the seven objects using **seven separate SNMP GET operations**.

For each returned object, display:

```
OID | data type | value
```

Measure the total time required to retrieve all seven objects.

Python's `time.perf_counter()` can be used to measure elapsed time:

```
import time

start = time.perf_counter()

# Operations to be measured

elapsed = time.perf_counter() - start
print(f"Elapsed time: {elapsed:.4f} s")
```

! Important

The seven requests must be performed **sequentially** in this experiment.
Do not use `asyncio.gather()` yet.

6.2.2 Experiment 2 — Several VarBinds in one GET

Modify the program so that all seven objects are requested using **a single SNMP GET operation**.

A GET operation can contain several `ObjectType` arguments:

```
await get_cmd(
    # ...
    ObjectType(ObjectIdentity("OID_1")),
    ObjectType(ObjectIdentity("OID_2")),
    ObjectType(ObjectIdentity("OID_3")),
)
```

Display the returned objects and measure the total execution time again.

Compare the result with Experiment 1.

Answer the following:

1. How many SNMP request messages are generated in Experiment 1?
2. How many are generated in Experiment 2?
3. How many SNMP response messages are generated in each case?
4. Which approach is faster in your experiment?
5. Why?
6. Is requesting an arbitrarily large number of objects in a single GET always a good idea?

6.3 Verifying the Difference on the Network

Use a packet capture to repeat Experiments 1 and 2.

You do not need to analyse every packet in detail. Instead, verify the number of SNMP requests and responses generated in each experiment.

You should observe approximately:

Experiment 1

Manager		Agent
	GET sysDescr	>
<	RESPONSE	
	GET sysObjectID	>
<	RESPONSE	
	GET sysUpTime	>
<	RESPONSE	
	...	

Experiment 2

Manager		Agent
	GET	>
	sysDescr	
	sysObjectID	
	sysUpTime	
	...	

```
< RESPONSE
    multiple VarBinds
```

Relate the packet capture to the execution times measured by your Python program.

6.4 What Happens When an Object Does Not Exist?

We will now examine how SNMP handles errors.

Create a GET request containing several valid objects from the **system** group and one OID that does not exist.

For example, you may deliberately modify one of the valid OIDs.

First perform the request using **SNMPv1**.

In PySNMP, SNMPv1 can be selected using:

```
CommunityData("public", mpModel=0)
```

Record:

- `error_indication`;
- `error_status`;
- `error_index`;
- the returned VarBinds.

Now repeat exactly the same request using **SNMPv2c**:

```
CommunityData("public", mpModel=1)
```

Compare the results.

6.4.1 SNMPv1 vs SNMPv2c

1. What happens to the complete request when one of the requested objects does not exist in SNMPv1?
2. Can you determine which VarBind caused the error?
3. What happens with SNMPv2c?
4. Are the valid objects still returned?
5. How is the nonexistent object represented?

Explain why the SNMPv2c behaviour is more useful when a manager requests several objects in a single operation.

6.5 Sequential Requests vs Concurrent Requests

Experiment 1 deliberately performed the seven GET operations sequentially:

```
GET 1 → wait → GET 2 → wait → GET 3 → wait → ...
```

However, SNMP requests are primarily **I/O-bound operations**: most of the time the program is waiting for network responses.

Create a third version in which the seven individual GET operations are started concurrently using `asyncio.gather()`.

Conceptually:

```
results = await asyncio.gather(  
    get_one(OID_1),  
    get_one(OID_2),  
    get_one(OID_3),  
    # ...  
)
```

Measure the execution time.

You now have three different strategies:

Strategy	SNMP requests	Execution
Individual GETs	7	Sequential
Multiple VarBinds	1	Single request
Individual GETs + <code>gather()</code>	7	Concurrent

Compare the three measured times.

Why can concurrent individual GETs be significantly faster than sequential GETs even though both approaches generate the same number of SNMP messages?

Does concurrency eliminate the network overhead of sending multiple SNMP requests?

Which of the three approaches would you choose for retrieving several objects from **one device**?

Would your answer necessarily be the same if you were monitoring **many different devices**?

i Note

This distinction will become important later in the lab.

When monitoring a single device, grouping several objects into one SNMP request can

reduce protocol overhead.

When monitoring many independent devices, asynchronous execution allows the manager to communicate with several agents without waiting for each operation to finish before starting the next one.

7 Navigating the MIB: GETNEXT and WALK

So far, every GET operation required the manager to know the exact OID of the object to retrieve.

But what happens if the manager does not know all the objects available below a particular OID?

SNMP provides the **GETNEXT** operation for this purpose.

7.1 Using GETNEXT

A GETNEXT request does not ask for the value of the specified OID itself. Instead, it asks the agent for the **next OID in lexicographical order**.

For example, consider part of the MIB-II **system** subtree:

```
1.3.6.1.2.1.1
```

```
1.3.6.1.2.1.1.1.0    sysDescr.0
1.3.6.1.2.1.1.2.0    sysObjectID.0
1.3.6.1.2.1.1.3.0    sysUpTime.0
1.3.6.1.2.1.1.4.0    sysContact.0
1.3.6.1.2.1.1.5.0    sysName.0
...
```

If the manager asks for the object following:

```
1.3.6.1.2.1.1.1.0
```

the agent should return:

```
1.3.6.1.2.1.1.2.0
```

Try it from the command line:

```
snmpgetnext -v2c -c public <MIKROTIK_IP> 1.3.6.1.2.1.1.1.0
```

Now try:

```
snmpgetnext -v2c -c public <MIKROTIK_IP> 1.3.6.1.2.1.1.2.0
```

and observe the returned OID.

What information must the manager know to perform a GET?

What information must it know to perform a GETNEXT?

Why can GETNEXT be useful when the manager does not know the complete contents of a MIB subtree?

7.2 Building a WALK from GETNEXT

There is no WALK PDU in the SNMP protocol.

An SNMP walk is obtained by repeatedly asking the agent for the next object:

```
start OID
```

```
    GETNEXT
```

```
returned OID
```

```
        GETNEXT
```

```
            returned OID
```

```
                ...
```

The OID returned by one request becomes the starting OID for the next request.

The process continues until the returned OID no longer belongs to the subtree being explored.

7.2.1 Experiment — Implement your own walk

Create:

```
my_walk.py
```

Your program must receive a starting OID and use repeated GETNEXT operations to display every object below that OID.

Test it using the MIB-II `system` subtree:

```
1.3.6.1.2.1.1
```

The output should contain lines similar to:

```
1.3.6.1.2.1.1.1.0  OctetString  ...
1.3.6.1.2.1.1.2.0  ObjectIdentifier ...
1.3.6.1.2.1.1.3.0  TimeTicks    ...
...
```

Your program must stop automatically when the returned OID is no longer below:

```
1.3.6.1.2.1.1
```

Warning

Do not simply perform a fixed number of GETNEXT operations.
A network management application cannot assume how many objects an agent implements below a subtree. Your program must determine when it has left the requested subtree.

Suppose your program starts at:

```
1.3.6.1.2.1.1
```

and eventually receives:

```
1.3.6.1.2.1.2....
```

Why should the walk stop even though the agent has successfully returned another object?

7.3 Comparing with `snmpwalk`

Now execute:

```
snmpwalk -v2c -c public <MIKROTIK_IP> 1.3.6.1.2.1.1
```

Compare its output with your Python program.

Both should traverse essentially the same part of the MIB.

i Note

`snmpwalk` is a convenient manager-side operation. It should not be confused with an SNMP protocol PDU.

Understanding this distinction is important:

SNMP protocol operation	Manager-side operation
GET	GET
GETNEXT	WALK
GETBULK	
SET	
RESPONSE	
...	

8 Working with SNMP Tables

Scalar objects such as `sysName.0` represent a single value.

Network management, however, frequently involves collections of similar objects. A router may have many interfaces, routes, neighbours, or addresses.

SNMP represents this type of information using **tables**.

8.1 The Interface Table

MIB-II provides information about the network interfaces of a device below the **interfaces** subtree:

```
1.3.6.1.2.1.2
```

Explore it:

```
snmpwalk -v2c -c public <MIKROTIK_IP> 1.3.6.1.2.1.2
```

The interface table contains information such as:

Object	Meaning
ifIndex	Interface identifier
ifDescr	Interface description
ifType	Interface type
ifMtu	Maximum transmission unit
ifSpeed	Estimated interface bandwidth
ifPhysAddress	Physical address
ifAdminStatus	Desired interface state
ifOperStatus	Current operational state
ifInOctets	Received octets
ifOutOctets	Transmitted octets

Unlike scalar objects, table objects have an **index** identifying the row.

For example:

```
ifDescr.1 = "ether1"  
ifDescr.2 = "ether2"  
ifDescr.3 = "ether3"
```

can be understood as:

	ifIndex		
	1	2	3
ifDescr	ether1	ether2	ether3
ifType	...	...	...
ifSpeed	...	...	...
ifOperStatus	...	...	...
ifInOctets	...	...	...
ifOutOctets	...	...	...

The index therefore connects values belonging to the **same interface**.

8.2 Building an Interface Inventory

Create:

```
interfaces.py
```

Use SNMP to discover the interfaces available on your MikroTik.

For each interface, display at least:

INDEX	NAME	TYPE	ADMIN	OPER
1	ether1	...	up	up
2	ether2	...	up	up
3	ether3	...	down	down
...				

Do not assume that the MikroTik has a fixed number of interfaces or that their indexes are consecutive.

Your program should obtain this information dynamically from SNMP.

Consider the following two objects:

```
ifDescr.4  
ifOperStatus.4
```

What does the .4 represent?

How does this differ from the .0 used in:

```
sysName.0
```

8.3 From Inventory to Telemetry

Now inspect the values of:

```
ifInOctets  
ifOutOctets
```

for one of the active interfaces.

Generate some network traffic and retrieve the values again.

Do `ifInOctets` and `ifOutOctets` represent:

- the current traffic rate,
- the number of bytes transferred during the last second, or
- cumulative counters?

What happens to their values while traffic crosses the interface?

Could you calculate a traffic rate if you obtained the same counter at two different times?

This observation will be the starting point for the next section, where we will turn SNMP counters into **network telemetry**.

9 From Counters to Telemetry

In the previous section, you observed that `ifInOctets` and `ifOutOctets` are **cumulative counters**.

They do not directly tell us the current traffic rate of an interface.

To obtain a rate, the manager must sample a counter at different times and calculate how much it has changed.

9.1 From a Counter to a Rate

Suppose that the value of `ifInOctets` is measured at two different times:

Time	Counter
t_1	C_1
t_2	C_2

The average incoming traffic rate during that interval is:

$$R_{\text{in}} = 8 \frac{C_2 - C_1}{t_2 - t_1}$$

where:

- C_1 and C_2 are counter values in octets;
- t_1 and t_2 are measured in seconds;
- the factor 8 converts octets into bits;
- R_{in} is expressed in bit/s.

The same calculation can be applied to `ifOutOctets`.

! Important

SNMP does not calculate the traffic rate for us. The agent exposes a **counter**. The manager samples that counter and derives the **rate**. This distinction between a raw measurement and a derived metric is fundamental in network monitoring and observability.

9.2 Measuring the Traffic Rate of an Interface

Create:

```
interface_rate.py
```

Your program must:

1. obtain the available interfaces from the MikroTik;
2. display them to the user;
3. allow the user to select one interface;
4. retrieve its input and output octet counters;
5. wait for a configurable sampling interval;
6. retrieve the counters again;
7. calculate the input and output traffic rates;
8. display the results in bit/s.

A possible execution could look like:

```
Available interfaces:
```

```
[1] ether1
[2] ether2
[3] ether3
```

```
Select interface: 1
```

```
Sampling interval [s]: 5
```

```
Interface: ether1
```

```
RX:  2.43 Mbit/s
```

```
TX:  817.32 kbit/s
```

If the sampling interval is 5 seconds, does the calculated value represent the instantaneous traffic rate at the moment of the second measurement?

What does it actually represent?

9.3 Sampling Interval

The **sampling interval** determines how often the monitoring system retrieves the counters.

Consider two extreme cases:

Short sampling interval

C1 C2 C3 C4 C5 C6

1s 1s 1s 1s 1s

More detail

More SNMP requests

Long sampling interval

C1 C2

60 s

Less detail

Fewer SNMP requests

Compare sampling intervals of:

1 s
10 s
60 s
5 min

What are the advantages and disadvantages of each?

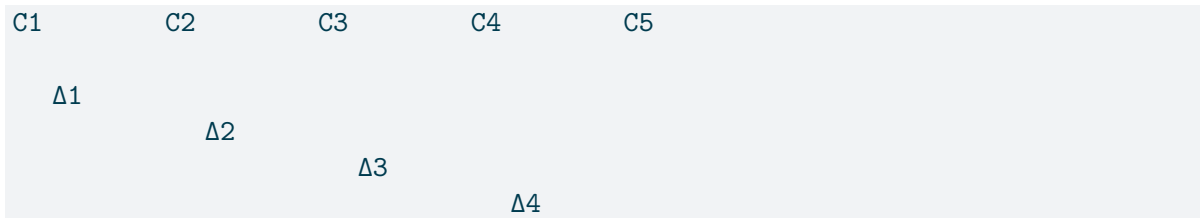
Would you use the same sampling interval to monitor 5 interfaces and 50,000 interfaces?

9.4 Continuous Monitoring

Modify `interface_rate.py` so that it continuously monitors the selected interface.

After the first measurement, you do not need to retrieve two completely new samples for every rate calculation.

Instead:



Each new counter value becomes the previous value for the next calculation.

Display one measurement per sampling interval:

TIME	RX	TX
14:32:05	1.24 Mbit/s	340.1 kbit/s
14:32:10	2.81 Mbit/s	512.4 kbit/s
14:32:15	950.2 kbit/s	1.13 Mbit/s
...		

Stop the monitor using **Ctrl+C**.

9.5 Counter Size Matters

The original MIB-II interface table defines:

```
ifInOctets
ifOutOctets
```

as 32-bit counters.

A 32-bit unsigned counter can represent values from:

$$0 \text{ to } 2^{32} - 1$$

After reaching its maximum value, the counter wraps around.

Consider a continuously busy **1 Gbit/s** interface.

Calculate approximately how long it would take a 32-bit octet counter to wrap around at 1 Gbit/s.

Is this a problem if the monitoring system samples the interface every 5 minutes?

For higher-capacity interfaces, IF-MIB provides 64-bit counters:

```
ifHCInOctets  
ifHCOutOctets
```

where **HC** stands for *High Capacity*.

Whenever they are available, these counters are generally preferable for traffic monitoring.

i Note

A robust monitoring application should not simply assume that:

```
new_counter >= old_counter
```

A counter may become smaller because of:

- counter wraparound;
- device or interface restart;
- agent restart.

Production monitoring systems must account for these situations.

9.6 Using High-Capacity Counters

Check whether your MikroTik exposes:

```
ifHCInOctets  
ifHCOutOctets
```

for the selected interface.

Modify your monitor to use the 64-bit counters when they are available.

What ASN.1/SNMP data type is returned for `ifInOctets`?

What type is returned for `ifHCInOctets`?

How many bits does each counter use?

9.7 Interface Utilization

A traffic rate becomes easier to interpret when it is compared with the capacity of the interface.

If the nominal interface speed is S , the utilization can be calculated as:

$$U_{\text{in}} = 100 \frac{R_{\text{in}}}{S}$$

and:

$$U_{\text{out}} = 100 \frac{R_{\text{out}}}{S}$$

Retrieve the appropriate interface speed information through SNMP and extend your program to display utilization.

For example:

TIME	RX	RX UTIL.	TX	TX UTIL.
14:32:05	1.24 Mbit/s	1.24%	340.1 kbit/s	0.34%
14:32:10	2.81 Mbit/s	2.81%	512.4 kbit/s	0.51%
14:32:15	950.2 kbit/s	0.95%	1.13 Mbit/s	1.13%

Does the nominal speed reported by an interface necessarily represent the amount of bandwidth actually available to an application?

Why is interface utilization nevertheless a useful management metric?

10 Monitoring More Than One Interface

So far, the program monitors a single interface.

A network management system normally monitors many interfaces and many devices.

Suppose that retrieving the counters from one interface takes approximately 100 ms.

A purely sequential monitor could behave like:

```
interface 1    100 ms
interface 2      100 ms
interface 3        100 ms
interface 4          100 ms
...
```

As the number of monitored objects increases, waiting for every network operation sequentially becomes inefficient.

10.1 Concurrent Polling with `asyncio`

Modify your program so that it can monitor several interfaces concurrently.

Use the asynchronous concepts introduced in the Python class:

```
await asyncio.gather(
    poll_interface(...),
    poll_interface(...),
    poll_interface(...),
)
```

Conceptually:

Sequential

```
A: request      response
                B: request  response
                        C: request  response
```

Concurrent

```
A: request      response
B: request      response
C: request      response
```

The program can use the time spent waiting for one network response to process or wait for others.

Does `asyncio` make the MikroTik answer an individual SNMP request faster?

If not, why can it significantly reduce the total time required to poll many independent objects or devices?

10.2 Final Monitor

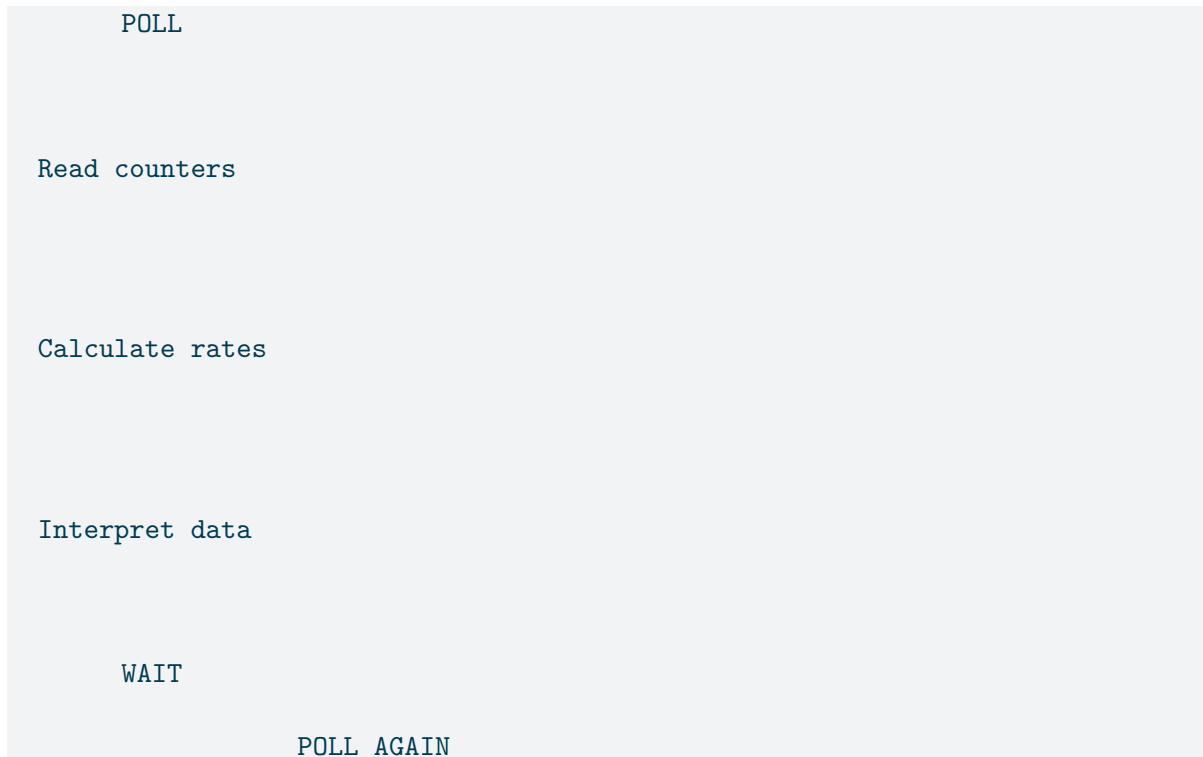
Your final program for this section should allow the user to select one or more interfaces and continuously display:

- interface name;
- operational status;
- input traffic rate;
- output traffic rate;
- input utilization;
- output utilization.

Use high-capacity counters when available.

The program should update the measurements periodically without requiring user interaction.

At this point, you have built the fundamental data collection loop used by a network monitoring system:



In later labs, the same general idea will be extended from individual SNMP measurements to a complete telemetry and observability platform.

11 Automatic Device Discovery

Until now, we have assumed that the network manager already knows the IP address of the device to monitor.

In a real network management system, this information may not be known in advance. Management platforms therefore provide **device discovery** mechanisms to find devices and build an inventory automatically.

A simplified discovery process can be represented as:



In this section, you will implement a simplified version of this process.

11.1 Discovering Hosts on a Local Network

If the manager is directly connected to an IPv4 LAN, **ARP** provides a useful mechanism for discovering active hosts.

The manager can send an ARP request asking:

```
Who has 192.168.153.1?  
Who has 192.168.153.2?  
Who has 192.168.153.3?  
...
```

A device using one of those addresses can reply with its MAC address:

Manager	Device
ARP: Who has 192.168.153.10?	>
< ARP: 192.168.153.10 is at MAC	

This provides two useful pieces of inventory information:

IP address	MAC address
------------	-------------

Note

ARP discovery only works for devices in the same Layer-2 broadcast domain. Routers do not normally forward ARP requests. Therefore, a management platform cannot use ARP alone to discover arbitrary remote networks.

11.2 ARP Discovery with Scapy

For this experiment, we will use **Scapy**, a Python library for creating, sending, receiving, and inspecting network packets.

We will only use a very small part of Scapy. The objective of this lab is not to learn the complete library, but to use it to implement the discovery mechanism discussed above.

Install Scapy inside your virtual environment:

```
pip install scapy
```

11.2.1 Building Packets with Scapy

Scapy represents network protocols as Python objects.

For example:

```
from scapy.all import ARP

packet = ARP(
    pdst="192.168.153.10"
)
```

creates an ARP packet asking about the IPv4 address 192.168.153.10.

You can inspect the packet using:

```
packet.show()
```

Try it.

Find the following fields in the output:

- operation (**op**);
- target protocol address (**pdst**);
- sender protocol address (**psrc**);
- target hardware address (**hwdst**).

What values has Scapy automatically selected for the fields that you did not specify?

11.2.2 Ethernet and ARP

ARP requests are transmitted inside Ethernet frames.

An ARP request intended for all hosts on the LAN therefore uses the Ethernet broadcast address:

```
ff:ff:ff:ff:ff:ff
```

In Scapy, an Ethernet frame can be created with:

```
from scapy.all import Ether

ethernet = Ether(dst="ff:ff:ff:ff:ff:ff")
```

Scapy uses the / operator to encapsulate one protocol inside another:

```
packet = ethernet / ARP(pdst="192.168.153.10")
```

This represents:

```
Ethernet
dst = ff:ff:ff:ff:ff:ff

    ARP

    Who has
    192.168.153.10?
```

Inspect the complete packet:

```
packet.show()
```

Notice that / does **not** mean division in this context. Scapy overloads the operator to construct a protocol stack.

11.2.3 Sending and Receiving Packets

Scapy provides several functions for sending packets.

Because we are working directly with Ethernet frames, we will use `srp()`:

```
from scapy.all import srp

answered, unanswered = srp(
    packet,
    timeout=2,
    verbose=False
)
```

`srp()` means approximately:

```
send and receive packets at Layer 2
```

and returns two collections:

```
answered  
unanswered
```

Each item in `answered` contains both:

```
packet sent → packet received
```

For an ARP response, the received packet provides the information we need:

```
for sent, received in answered:  
    print(received.psrc)  
    print(received.hwsrc)
```

where:

```
psrc    → source protocol address → IPv4 address  
hwsrc   → source hardware address → MAC address
```

Scapy and asyncio

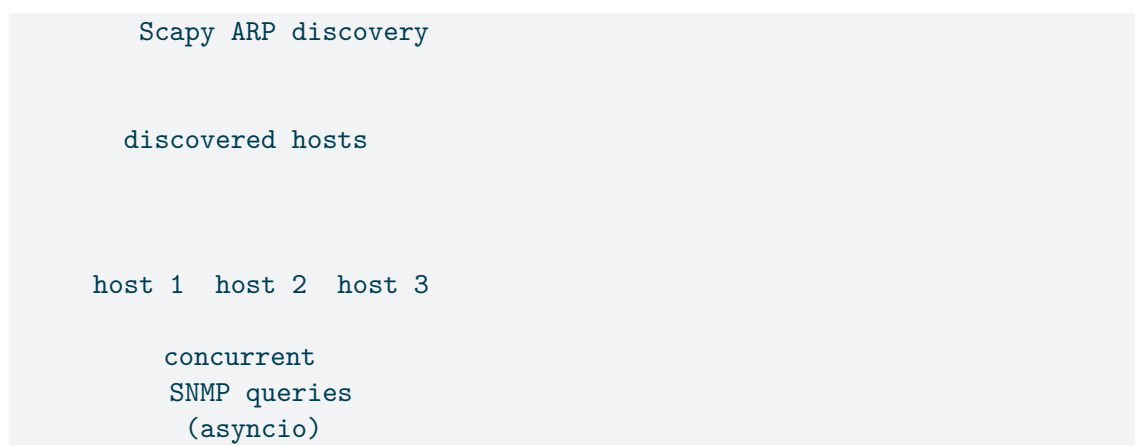
The `srp()` function used here is a **blocking operation**.
Calling it from an `async` function does not automatically make it asynchronous:

```
async def discover():  
    answered, unanswered = srp(...)
```

While `srp()` is running, the Python `asyncio` event loop cannot execute other tasks.
For this exercise, this is not a problem. Scapy sends the ARP probes and collects their responses as a single discovery operation:



We will therefore perform ARP discovery with Scapy first and then use `asyncio` for the SNMP queries:



This also illustrates an important Python concept:

`asyncio` provides concurrency only when the operations being called are asynchronous and yield control while they are waiting.

11.2.4 Discovering a Complete Subnet

Scapy also accepts a network in CIDR notation as the destination of an ARP request.

For example:

```
ARP(pdst="192.168.153.0/24")
```

represents ARP requests for the addresses in that subnet.

Therefore, the basic discovery operation can be constructed as:

```

from scapy.all import ARP, Ether, srp

network = "192.168.153.0/24"

packet = (
    Ether(dst="ff:ff:ff:ff:ff:ff")
    / ARP(pdst=network)
)

answered, unanswered = srp(
    packet,
    timeout=2,
    verbose=False
)

```

Before continuing, make sure you can explain each part of:

```
Ether(dst="ff:ff:ff:ff:ff:ff") / ARP(pdst=network)
```

In particular:

1. Why is the Ethernet destination address a broadcast address?
2. Why can the ARP destination contain a /24 network rather than a single address?
3. What does / mean in Scapy?
4. Why are we using `srp()` rather than simply sending the packets?

11.2.5 Exercise — Build the Host Inventory

Create:

```
discover.py
```

Using the previous example as a starting point, process the responses and produce a table containing the discovered IP and MAC addresses:

IP ADDRESS	MAC ADDRESS
192.168.153.1	48:a9:8a:...
192.168.153.4	02:42:ac:...
192.168.153.17	bc:24:11:...
...	

Do not copy the example blindly. You should understand how the Ethernet frame, the ARP request, `srp()`, and the returned responses work before constructing the inventory.

11.3 ARP Is Not the Only Discovery Mechanism

Network discovery does not rely on a single universal mechanism.

Depending on the network and the information available, management systems may combine different sources such as:

```
ARP / neighbour tables
ICMP probes
SNMP
LLDP/CDP
routing information
DHCP information
DNS
traffic observation
...
```

For example, ICMP Echo Requests can also be used to probe IP addresses:

Manager	Device
ICMP Echo Request	>
< ICMP Echo Reply	

However:

Warning

No response to an ICMP Echo Request does **not** necessarily mean that no device exists at that address.

ICMP may be filtered while other protocols, including SNMP, remain reachable.

For this lab, **ARP will be used for initial discovery** because the devices being discovered are on the local laboratory network.

11.4 From a Host to a Managed Device

Finding an IP address does not tell us whether the device can be managed using SNMP.

For every host discovered using ARP, send an SNMPv2c GET request for:

```
sysName.0
```

using the community:

```
public
```

If the device provides a valid SNMP response, classify it as an **SNMP-enabled device**.

Your inventory should now distinguish:

IP ADDRESS	MAC ADDRESS	SNMP
192.168.153.1	48:a9:8a:...	yes
192.168.153.4	02:42:ac:...	no
192.168.153.17	bc:24:11:...	yes

A host was discovered using ARP but does not answer the SNMP request.

Can you conclude that the device does not support SNMP?

What other explanations are possible?

11.5 Identifying SNMP Devices

For each device that responds to SNMP, retrieve:

```
sysName.0  
sysDescr.0  
sysObjectID.0
```

Extend the inventory:

```
192.168.153.1  
  MAC:      48:a9:8a:..  
  SNMP:     yes  
  Name:     router01  
  Description: Router0S ...  
  Object ID: 1.3.6.1.4.1....  
  
192.168.153.4  
  MAC:      02:42:ac:..  
  SNMP:     no
```

The three SNMP objects provide different information:

- `sysName` provides the configured name of the device;
- `sysDescr` provides a textual description of the system;
- `sysObjectID` identifies the type of device according to the vendor's object hierarchy.

Which of these objects would be the most appropriate for software that needs to automatically determine what type of device it has discovered?

Why is parsing `sysDescr` text generally less robust?

11.6 Enterprise OIDs

Inspect the `sysObjectID` returned by your MikroTik.

Vendor-specific objects are located below:

```
1.3.6.1.4.1
```

This part of the OID tree is known as:

```
private.enterprises
```

Each registered organization receives an **enterprise number**.

Conceptually:

```
1.3.6.1.4.1
    enterprises
        vendor A
        vendor B
        vendor C
        products...
    ...
```

Identify the enterprise number contained in the `sysObjectID` returned by your MikroTik.

i Note

This is one of the mechanisms that a network management platform can use after discovery to decide which vendor-specific MIBs, monitoring templates, or collection rules should be applied to a device.

11.7 Why Sequential Discovery Does Not Scale

Suppose several devices have been discovered:

```
192.168.153.1
192.168.153.2
192.168.153.3
...
```

A sequential SNMP probe behaves approximately as:

```
Device 1: request      wait      response
Device 2:                                     request  wait      response
Device 3:                                                         ...
```

This becomes particularly inefficient when a device does **not** respond.

The manager must wait for the SNMP timeout before moving to the next device.

Assume an SNMP timeout of 1 second.

Approximately how long could a sequential scan of 100 addresses take if none of them responds?

What happens if those requests can be performed concurrently?

11.8 Concurrent SNMP Discovery

Modify the SNMP phase of `discover.py` to probe discovered hosts concurrently using `asyncio`.

Conceptually:

```
results = await asyncio.gather(
    probe_snmp("192.168.153.1"),
    probe_snmp("192.168.153.2"),
    probe_snmp("192.168.153.3"),
    # ...
)
```

The discovery process now becomes:

ARP discovery

IP1 IP2 IP3 IP4 IP5 ...

concurrent SNMP

SNMP responders

Device information

Measure the time required for:

1. sequential SNMP probing;
2. concurrent SNMP probing.

Compare the results.

Why is concurrency particularly valuable when some devices do not respond?

Does concurrent discovery generate less network traffic than sequential discovery?

What resource or network problems could arise if a management platform started tens of thousands of discovery requests simultaneously?

11.9 Building the Device Inventory

Your final `discover.py` program should automatically:

1. discover active devices on the local network;
2. obtain their IP and MAC addresses;
3. determine which devices respond to SNMP;
4. retrieve `sysName`, `sysDescr`, and `sysObjectID` from SNMP-enabled devices;
5. display the resulting inventory.

A possible result is:

```
Network discovery: 192.168.153.0/24
```

```
Discovered: 6 devices  
SNMP-enabled: 2 devices
```

IP ADDRESS	MAC ADDRESS	SNMP	NAME	DESCRIPTION
192.168.153.1	48:a9:8a:...	yes	router01	RouterOS ...
192.168.153.4	02:42:ac:...	no	-	-
192.168.153.8	bc:24:11:...	yes	switch01	...
...				

You have now implemented a simplified version of an important function provided by many network management platforms:

```
DISCOVERY  
  ↓  
IDENTIFICATION  
  ↓  
INVENTORY  
  ↓  
MONITORING
```

The next step after discovering and identifying a device would be to determine **what should be monitored on that device**.

12 Modifying Managed Objects: SNMP SET

Until now, all our SNMP operations have been **read operations**:

```
GET / GETNEXT / WALK
```

```
Manager reads information from the agent
```

SNMP can also modify managed objects using the **SET** operation:

Manager	Agent
SET request	>

< RESPONSE

However, only objects defined as writable in their MIB can be modified.

12.1 Read and Write Access

For the previous exercises, the MikroTik SNMP agent was configured with a read-only community:

```
public
```

This is sufficient for monitoring, but it should not allow configuration changes.

Try to modify `sysLocation.0` using the read-only community.

For example, with the Net-SNMP command-line tools:

```
snmpset -v2c -c public <MIKROTIK_IP> \  
1.3.6.1.2.1.1.6.0 s "Network Management Lab"
```

Here:

```
1.3.6.1.2.1.1.6.0 → sysLocation.0  
s → OCTET STRING  
"Network Management Lab" → new value
```

Observe the result.

Was the operation accepted?

Why should a community used by a monitoring system normally have read-only access?

12.2 Enabling Write Access

Configure a **separate SNMP community** on the MikroTik with write access.

Do not modify the read-only `public` community.

Use the write community only for this part of the lab.

 Warning

A read-write SNMP community provides the ability to modify writable managed objects on the device.

It should therefore not be used for routine monitoring.

Repeat the SET operation using the write community.

Then verify the result independently with a GET:

```
snmpget -v2c -c public <MIKROTIK_IP> \  
1.3.6.1.2.1.1.6.0
```

The complete operation is therefore:



Why is it useful to verify the result using a separate GET rather than assuming that the configuration has changed because a SET request was sent?

12.3 Performing a SET with PySNMP

Create:

```
set_location.py
```

The program must receive a new location as an argument:

```
python set_location.py "Lab 1 - EINA"
```

and modify:

```
sysLocation.0
```

using SNMP.

After performing the SET, retrieve the object again and display its new value.

Your program should produce output similar to:

```
Previous location: ...  
Setting location...  
New location: Lab 1 - EINA
```

Compare GET and SET in your Python programs.

Which elements are common to both operations?

What changes?

12.4 Inspecting an SNMP SET

Capture one successful SET operation.

Identify:

- the SNMP version;
- the community;
- the PDU type;
- the OID being modified;
- the new value;
- the agent response.

Pay particular attention to the community string.



A Security Problem

In SNMPv1 and SNMPv2c, the community string is transmitted without encryption. Anyone able to capture the traffic may be able to obtain the community used for the SET operation.

A captured read-write community is particularly dangerous because it may allow an attacker to modify the device configuration.

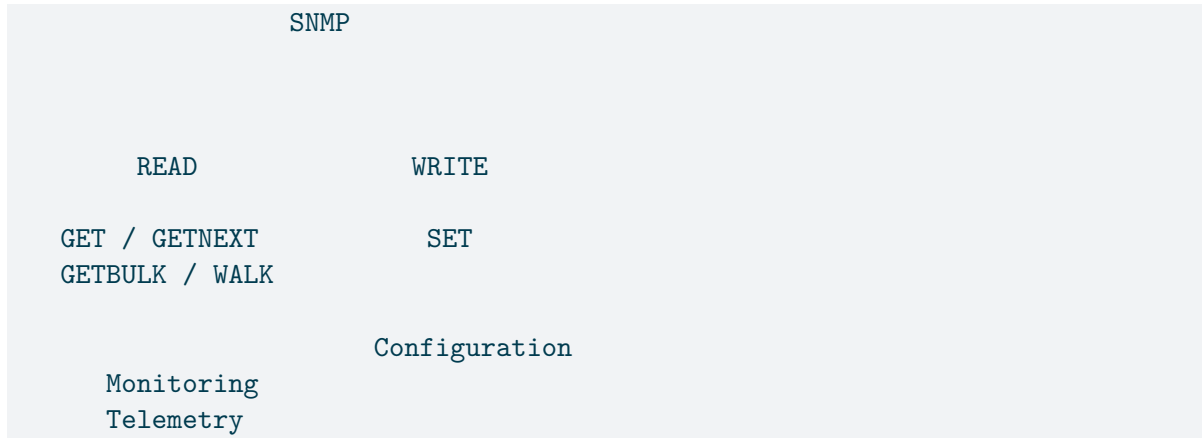
Compare the packet capture of your earlier SNMP GET using `public` with this SET operation.

Can you identify both community strings in the captured packets?

Would you consider SNMPv2c appropriate for management traffic crossing an untrusted network?

12.5 Monitoring vs Configuration

SNMP can therefore be used for both:



In this course, our main use of SNMP will be **monitoring and telemetry**.

Modern network automation frequently uses other mechanisms for configuration, while SNMP remains widely used for retrieving operational state and metrics.

Nevertheless, understanding SET is important because it demonstrates that the SNMP management model was designed for both **monitoring and control**.

13 Securing SNMP: SNMPv3

In the previous section, you captured an SNMPv2c SET operation.

The packet contained not only the object being modified and its new value, but also the **community string** used to authorize the operation.

This creates an obvious security problem.

If someone can capture the credential that allows us to modify the router, how can SNMP be used securely?

13.1 The Problem with Community-Based SNMP

SNMPv1 and SNMPv2c use a **community string** as a simple access-control mechanism.

Conceptually:

```
Manager      "public"      Agent
              GET
Manager      "write-community" Agent
              SET
```

The community identifies the access rights associated with the request.

However, the community itself is transmitted without encryption.

As you observed in the packet capture:

```
Network observer

SNMPv2c packet

Community: public
PDU: GET
OID: ...
```

and, even more critically:

```
SNMPv2c packet

Community: *****
PDU: SET
OID: sysLocation.0
Value: ...
```

What could an attacker do after capturing a valid read-write community?

Would choosing a very long and random community string solve the packet-capture problem?
Why?

14 SNMPv3 Security Model

SNMPv3 introduces a different security model.

Instead of identifying access using only a community string, SNMPv3 can provide:

- **authentication** — verify who sent the message;
- **integrity** — detect whether the message was modified;
- **privacy** — encrypt the protected part of the SNMP message.

SNMPv3 defines three commonly used security levels:

Security level	Authentication	Encryption
<code>noAuthNoPriv</code>	No	No
<code>authNoPriv</code>	Yes	No
<code>authPriv</code>	Yes	Yes

The names can be read as:

`noAuthNoPriv`

no privacy
no authentication

`authNoPriv`

no privacy
authentication

`authPriv`

privacy
authentication

Which security level would you choose for management traffic crossing an untrusted network?

What information is still potentially visible when using `authNoPriv`?

14.1 Configuring an SNMPv3 User

Configure an SNMPv3 user on your MikroTik for this experiment.

Use:

```
Security level: authPriv
```

and the authentication and privacy credentials assigned for your laboratory environment.

Warning

Do not reuse personal passwords.

The credentials used in this exercise are laboratory credentials and should only be used inside the CyberRange.

Your configuration requires several pieces of information:

```
SNMPv3 user

    username

    authentication protocol
        authentication password

    privacy protocol
        privacy password
```

Verify that SNMPv3 access is enabled before continuing.

14.2 Your First SNMPv3 GET

Use the Net-SNMP command-line tools to retrieve:

```
sysUpTime.0
```

using SNMPv3 with authentication and privacy.

The command has the following general structure:

```
snmpget -v3 \  
-l authPriv \  
-u <USERNAME> \  
-a <AUTH_PROTOCOL> \  
-A '<AUTH_PASSWORD>' \  
-x <PRIV_PROTOCOL> \  
-X '<PRIV_PASSWORD>' \  
<MIKROTIK_IP> \  
1.3.6.1.2.1.1.3.0
```

where:

```
-v3    SNMP version 3  
-l     security level  
-u     username  
-a     authentication protocol  
-A     authentication password  
-x     privacy protocol  
-X     privacy password
```

Compare this command with the SNMPv2c equivalent:

```
snmpget -v2c -c public <MIKROTIK_IP> 1.3.6.1.2.1.1.3.0
```

What information has replaced the community string?

14.3 Inspecting SNMPv3 on the Network

Capture the SNMPv3 GET request and its response.

Compare them with the SNMPv2c GET captured earlier.

Try to identify:

- the SNMP version;
- the SNMPv3 username;
- the authentication-related information;
- the encrypted portion of the message.

Then try to locate:

```
sysUpTime.0
```

and its returned value in the packet capture.

Can you read the requested OID as easily as you could with SNMPv2c?

Can you read the returned value?

Can you find the authentication password in the packet?

Can you find the privacy password?

Can you find the SNMPv3 username?

! Important

Encryption does not mean that the complete network packet becomes invisible. An observer can still see information required to transport and process the packet, such as IP addresses, UDP ports and some SNMPv3 metadata. With `authPriv`, however, the protected SNMP scoped PDU is encrypted.

14.4 SNMPv3 with PySNMP

Create:

```
get_v3.py
```

Modify one of your previous GET programs so that it retrieves:

```
sysUpTime.0
```

using SNMPv3 with `authPriv`.

In PySNMP, SNMPv3 user-based security is configured using `UsmUserData` instead of `CommunityData`.

Conceptually:

```
CommunityData(...)
```

becomes:

```

UsmUserData(
    username,
    authKey=...,
    privKey=...,
    authProtocol=...,
    privProtocol=...
)

```

Use the authentication and privacy protocols configured on your MikroTik.

Look at the rest of your previous PySNMP GET program.

Apart from the security configuration, how much of the SNMP operation has changed?

What does this tell you about the separation between:

what we want to retrieve

and:

how the SNMP communication is secured

15 Comparing SNMPv2c and SNMPv3

You have now retrieved the same managed object using both SNMPv2c and SNMPv3.

Complete the following comparison:

	SNMPv2c	SNMPv3 authPriv
Identity/access mechanism	Community	
Authentication		
Message integrity		
Encryption/privacy		
Credential visible in capture		
OID/value readable in capture		

If you were deploying a new network monitoring system today, which version would you choose when the devices support it?

Why might SNMPv2c nevertheless still be encountered in real networks?

16 What Have We Built?

At the beginning of this lab, we had only:

Linux container + MikroTik

You have progressively implemented the basic functions of a network management system:

NETWORK

DISCOVER

IDENTIFY

INSPECT
MIB/OIDs

MONITOR
counters

DERIVE
metrics

ACT
SET

SECURE SNMPv3

These operations are small-scale examples of functions performed by real network management and observability platforms.

The tools will change during the course, but many of the underlying engineering problems remain the same:

How do I discover what exists?

How do I identify it?

What information can I collect?

How often should I collect it?

How do I transform raw measurements into useful metrics?

How do I perform management operations safely?

How does this scale to many devices?